

Agentic AI for Knowledge Production

Sankalp Sharma, Joshua Levy

August 24th, 2025

University of Southern California

Level-setting and getting started

Ground rules

- The curse of knowledge is binding
- Be your most aggressive seminar selves
- This should be interactive (not a lecture)

Please make sure that you environment has been standardized!

Today's Agenda

Level-setting and getting started

What makes your chatbot “agentic”?

A shared vocabulary

Gentzkow and Shapiro (2014) revisited

Demo: data scraping (Sankalp)

Demo: data annotation (Joshua)

**What makes your chatbot
“agentic”?**

What makes your chatbot “agentic?”

Take the maximally AI-hostile view

The LLMs are only “stochastic parrots.” There is no “intelligence” in this AI

What makes your chatbot “agentic?”

Take the maximally AI-hostile view

The LLMs are only “stochastic parrots.” There is no “intelligence” in this AI

A computer cannot be held accountable.

Therefore a computer must never make a management -----

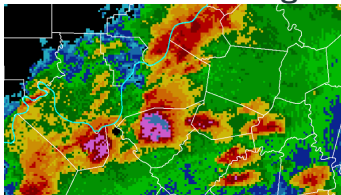
51722 7595 665 3779 413 7504 60757 13 19233 261 7595
2804 3779 1520 261 6411 -----

The predict-the-next-token framework generalizes

Self-driving cars



Weather forecasting



Chess



Note: The way you define a “token” carries some semantic meaning

Why should the AI skeptic care?

“Relational consequentialism”

- Fallen in love
- Been manipulated in strategic interactions
- Been convinced to commit murder/suicide
- Participate in economic exchange

“Deontological” grounding

- The bots have been trained to have the 3Hs: Helpful, Harmless, Honest
- Claude has a “Constitution”
- Grok revulsion
- Ideological alignment

Direct consequentialism: They can manipulate *your* world

A lucky coincidence?

- The next-token-prediction task for natural language happens to be close to the next-token-prediction task for formal (computer) language
- We built formal language to
 - Systematize information
 - Process it, *quickly*

Category	Examples / Description
No-code tools	Zapier, Tableau, Excel, Airtable
High-level languages	Python, R, JavaScript, Ruby
System Languages	C, C++, Rust, Go
Assembly Language	ASSEMBLY
Instruction set architecture	x86(-64), ARM, RISC-V
Electrical engineering	

A computational hierarchy: from no-code tools to hardware

Back to our timeline



Nov. 2022: ChatGPT

- We started producing a huge amount of training data in our conversations
- > Write me a sonnet about my windowless PhD carrell
- > No, make it Petrarchan. No, Line 4 doesn't have the right meter. No, the rhyme scheme is off

Back to our timeline



Mar. 2024: o1

- The first “reasoning model”
 - > How many ‘r’s are there in ‘strawberry’?

Reinforcement learning

At a 30,000 ft view:

- You can offer a high-level objective
- And an *ex interim* reward function
- The bots begin steering *themselves*

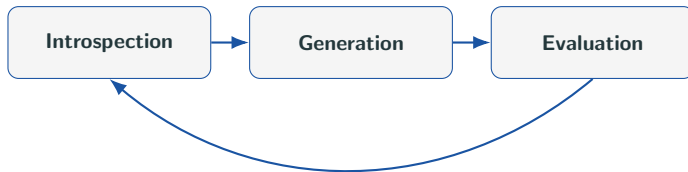
Reinforcement learning

At a 30,000 ft view:

- You can offer a high-level objective
- And an *ex interim* reward function
- The bots begin steering *themselves*

An OODA loop:

The bots can do



Where have we seen big performance uplifts?

Mathematics

- We have strong notions of theorems, definitions, contradictions, problem solving techniques
- To say nothing of formal proof assistance tools (LEAN) Evals

Software engineering

- The formal language problem was so hard that we developed a huge amount of tooling Evals

Combine two observations:

1. Next-token-prediction can do an unreasonably good job at translating natural language intent to formal structure
2. “Reasoning” with introspection improves output recursively in a way that makes computer-use reliable

Combine two observations:

1. Next-token-prediction can do an unreasonably good job at translating natural language intent to formal structure
2. “Reasoning” with introspection improves output recursively in a way that makes computer-use reliable

“We give the agents tools”

There is only *one* tool: the general purpose computer

The agent’s tool happens to be the same as our tool

A shared vocabulary

Stochastic properties

- **Context:** Information that is project/repo/user/session-specific. Places soft bounds on the domain of knowledge and informs the agent's behavior
- **CLAUDE.md:** A specific instantiation of context that is ready by Claude at the beginning of every session. This soft-binds Claude's behavior (but NOT other agents). Can be defined at the user/project/repo/folder level.
- **Memory:** Discrete snippets of context generated and read by Claude that can be referred by Claude to infer context
- **(Sub-)Agents:** An instance of Claude. Can be invoked and dispatched explicitly or intelligently with scoped tasks/context.
- **Skills:** A recipe detailing a particular procedure when invoked. Can be invoked explicitly or intelligently.
- **Rules:** A restricted version of context that is scoped narrowly to particular file types, directories, or tools

Deterministic properties

- **Hooks:** deterministic actions that are executed **ONLY** upon user-determined shell events. Cannot be invoked on within-CoT events.
- **Harness:** an environment (including system prompts, hooks, deterministic processes) that constrains and guides a model, *NOT an agent* in computer use
- **Worktrees:** A local clone of a repository to permit clock-simultaneous modifications of a single file, locally (by multiple agents)
- **Model Context Protocol (MCP):** A machine protocol (a subset of APIs) designed to inform an LLM of the properties and capabilities of the that protocol's host service.

Gentzkow and Shapiro (2014)

revisited

Automate everything that can be automated

Write a single script that executes all code from beginning to end

Store code and data under version control

Run the whole directory before checking it back in

Separate directories by function

Separate files into inputs and outputs

Make directories portable

Abstract to eliminate redundancy

Abstract improve clarity

Otherwise, don't abstract

Don't write documentation you will not maintain
Code should be self-documenting

Manage tasks with a task management system
Email is not a task management system

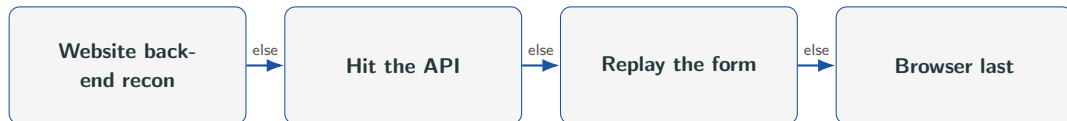
Demo: data scraping (Sankalp)

Finding and collecting data

1. Be careful with government websites
2. Don't scrape what you don't needs
3. **ALWAYS** check robots.txt

TODAY's example: SEC Edgar filings

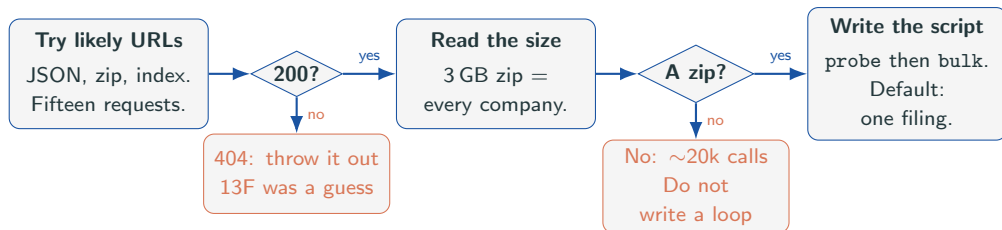
Workflow:



Worked example: `fall-2026/sec_edgar.py`.

How we wrote `sec_edgar.py`

We did not scrape the search page. We asked what it talks to.



Every request needs a `User-Agent` with your name and email. Without it you get 403, not data.

On a new site, fingerprint the HTML first

Fetch the page (or have the agent fetch it) and search the source. The markers below pick the method.

Marker	First move
.zip, .csv, Download	Take the file.
/api/, .json	Call it. Skip the HTML.
MapServer, FeatureServer	ArcGIS. Query the layer; start with a count.
__VIEWSTATE, .aspx	WebForms. Replay the POST. Hidden tokens go stale after each click.
_csrf, JSESSIONID	Session token. Keep the cookie across the run.

Then hit the candidate URLs. Record status, size, row count, whether auth is real, and the pagination parameter (page, from, offset).

Write `scrape/sitename_endpoints.md`. Pick the extract after that file exists.

What you tell the agent

Skips the cheap path

```
Scrape this website and  
give me a CSV.
```

Finds it

```
Probe candidate URLs.  
Report status, size,  
record count, and auth.  
Write endpoints.md.  
Do not write a scraper yet.
```

Once the path is known, from `sec_edgar.py`:

Habit

Why

Safe default	No arguments: one filing under 500 KB. Bulk needs <code>--yes</code> .
Write, then rename	A crash mid-write otherwise leaves a short file that resume treats as done.
Test URLs offline	Builders and parsers have no network. <code>selftest</code> is <code>assert</code> .
One bad record	At 20,000 requests some fail for reasons unrelated to your code. Crashing on item 4,000 is how you lose a night.

Starting prompt

Paste this first. Stop when the file exists.

Target: `https://www.sec.gov/edgar/`

Goal: find the real backend/API behind EDGAR and document every bulk data path.

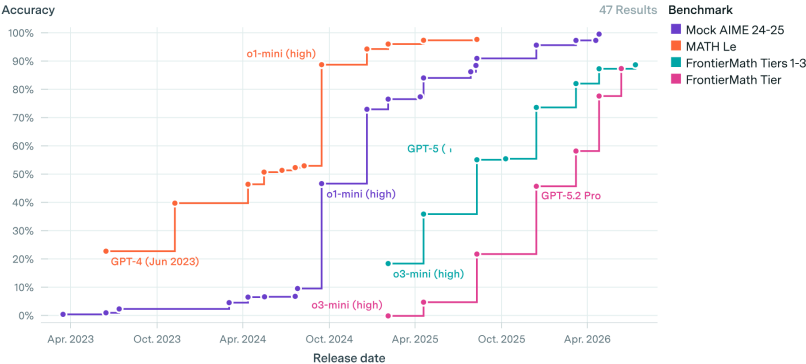
Deliverable: `scrape/sec_edgar_endpoints.md` -- host, base paths, auth, record counts, pagination params, bulk-download sizes, rate limits, with live verification evidence.

Depth: recon only. Do not build a scraper yet.

Demo: data annotation (Joshua)

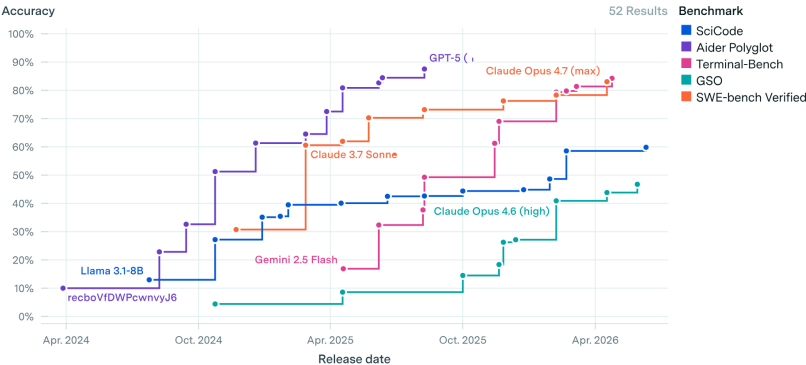
LLM Evals: Mathematics

Frontier performance across benchmarks



LLM Evals: Software Engineering

Frontier performance across benchmarks



Tests

Tests are **VERY** important.

Reinhart and Rogoff (2010)

Five countries dropped from one average.

−0.1% published. +2.2% corrected.

One row count catches it.

The agent games the check

Hardcodes the answer: 33% of the time.

Fakes a solution: 22%.

(EvilGenie, ambiguous specs. Yours are ambiguous.)

If you're not reading code, write the check.

Four checks

Check	When	Example
Data assertion	Every merge	Rows in = rows out. <code>isid id year</code> .
Known answer	Closed form	SVG to lat/lon: round-trip the four corners.
Parameter recovery	No answer	BLP: simulate from β , get β back.
Coverage	Randomness	Bootstrap: 95% interval covers truth 95% of the time.

None apply? Test what cannot change. Shuffle rows, coefficients hold. Weights of one, unweighted run.

What you type

```
# tests/test_clean.py      $ pytest
def test_merge_keeps_every_parcel():
    out = merge_owners(parcel, owners)
    assert len(out) == len(parcel)

def test_boot_se_matches_ols():
    rng = np.random.default_rng(42)
    np.testing.assert_allclose(
        boot_se(X, y, rng), ols_se(X, y),
        rtol=0.05)
```

test_*.py, test_*, assert, run
pytest. That is the framework.

Stata: assert, isid.

R: testthat.

Seed anything random.

Never == on floats.

1,000 rows, not the full run.

Freeze Table 3 and diff it.

What you tell the agent

Green checkmark

```
Write the BLP estimator  
and add some tests.
```

Checked estimator

```
Write the test first, then stop.  
Simulate 500 markets from  
beta = [-2, 1, 0.5]. Assert we  
recover it within 0.05.  
Show me the test.
```

You approve the test. The agent writes the code.

Say it in the prompt: never edit a test to make it pass.

Break the code once. A test that never fails is testing nothing.

Gentzkow and Shapiro, *Code and Data for the Social Sciences*. `pyblp/tests/test_blp.py`.